

Teach Module 46: Kafka in Production: Resilience and Observability

This note is based on Module 46's topic title from the bootcamp outline, but the teaching content is original and does not use the course material as the lesson source.

Module 46 Overview

Module 46 covers **Kafka in Production: Resilience and Observability**.

Kafka in development is often simple:

```
Producer sends message -> Consumer reads message -> Application works
```

Kafka in production is different.

Production Kafka systems must answer questions like:

- What happens when a consumer crashes?
- What if one message keeps failing forever?
- How do we know consumers are falling behind?
- Can we change message structure without breaking old services?
- How do we recover without losing or duplicating important events?

This module is about making Kafka-based systems reliable, observable, and safe to operate.

The Production Kafka Mindset

Kafka is not just a queue. It is an event log.

A topic stores events in order, divided into partitions. Consumers read from partitions and track their progress using offsets.

Example:

```
orders-topic  
Partition 0: offset 0, 1, 2, 3, 4  
Partition 1: offset 0, 1, 2, 3
```

A consumer does not remove messages from Kafka. It records:

```
I have successfully processed up to offset X.
```

That offset is extremely important.

If your consumer commits an offset too early, you may lose work.

If your consumer commits an offset too late, you may process the same message again.

So production Kafka code must be designed around this idea:

Processing may happen more than once, so your application should be safe when duplicate messages appear.

This is called **idempotency**.

Consumer Error Handling

Imagine a Kafka consumer receives this event:

```
{
  "orderId": 101,
  "customerId": 55,
  "amount": 49.99
}
```

The consumer tries to process the order, but the database is temporarily unavailable.

That failure is probably temporary. Retrying makes sense.

Now imagine the consumer receives this:

```
{
  "orderId": null,
  "amount": "not-a-number"
}
```

That message may never succeed because the data itself is bad. Retrying forever would block the consumer.

Production systems usually separate failures into two categories:

Failure Type	Example	Strategy
Transient failure	Database temporarily down	Retry
Permanent failure	Invalid message format	Send to dead-letter topic

Retry Strategy

A retry strategy says:

Try again, but not recklessly.

Bad retry behavior:

```
fail -> retry instantly -> fail -> retry instantly -> fail
```

That can overload your system.

Better retry behavior:

```
fail -> wait 1 second
fail -> wait 5 seconds
fail -> wait 30 seconds
fail -> give up and send to dead-letter topic
```

This delay pattern is called **backoff**.

A common production pattern is:

```
Main topic -> Consumer -> Retry topic -> Retry again -> Dead-letter topic
```

Example topic names:

```
orders
orders.retry.1m
orders.retry.5m
orders.DLT
```

Dead-Letter Topics

A dead-letter topic is where Kafka messages go when they cannot be processed successfully after reasonable attempts.

Example:

```
orders topic
|
v
OrderConsumer
|
|-- success -> commit offset
|
|-- repeated failure -> orders.DLT
```

A dead-letter message should usually include:

- Original message payload
- Original topic
- Partition and offset
- Error message
- Exception type
- Timestamp
- Service name that failed

Example dead-letter event:

```
{
  "originalTopic": "orders",
  "partition": 2,
  "offset": 9842,
```

```
"error": "Missing customerId",
"payload": {
  "orderId": 101,
  "customerId": null
}
}
```

Key idea:

A bad message should not stop the whole consumer group forever.

Dead-letter topics allow the system to keep moving while failed messages are preserved for investigation.

Poison Messages

A **poison message** is a message that repeatedly crashes or breaks a consumer.

Example:

```
{
  "eventType": "ORDER_CREATED",
  "orderId": 123,
  "items": null
}
```

If the consumer assumes `items` is always a list, this may throw a `NullPointerException`.

Without proper handling, the consumer may keep reading the same message, failing, restarting, and failing again.

The fix is structured handling:

```
try process message
if success:
  commit offset
if temporary failure:
  retry
if permanent failure:
  publish to DLT
  commit offset
```

Committing the offset after sending to the DLT is important. Otherwise, the consumer may keep re-reading the same poison message.

Consumer Lag

Consumer lag is one of the most important Kafka production metrics.

It means:

latest offset in Kafka - latest offset processed by consumer

Example:

```
Kafka latest offset:      10,000
Consumer committed offset: 9,200
Consumer lag:             800
```

That means the consumer is 800 messages behind.

Lag is not always bad. A short spike can be normal.

But growing lag is a warning sign.

Possible causes:

- Consumer is too slow
- Consumer instances crashed
- Database or API dependency is slow
- Too few partitions
- Too few consumer instances
- Messages are larger than expected
- A poison message is blocking progress
- Rebalancing is happening too often

A healthy system may look like:

```
Lag rises briefly, then returns close to zero.
```

An unhealthy system may look like:

```
Lag keeps increasing for 10, 20, 30 minutes.
```

That means the consumer is not keeping up.

Throughput

Throughput measures how many messages move through the system over time.

Producer throughput:

```
messages produced per second
```

Consumer throughput:

```
messages consumed per second
```

Example:

```
Producer writes: 2,000 messages/sec
Consumer reads:  1,200 messages/sec
```

That system will develop lag because messages are arriving faster than they are being processed.

To fix throughput problems, you may:

- Add more consumer instances
- Increase topic partitions
- Optimize database writes
- Batch processing
- Reduce expensive synchronous calls
- Improve message size
- Use async processing carefully

Important rule:

```
A consumer group can only process a topic in parallel up to the number of partitions.
```

If a topic has 3 partitions, then at most 3 consumers in the same group can actively consume from it.

```
3 partitions + 5 consumers = only 3 active consumers
```

The extra 2 consumers will sit idle.

Kafka Monitoring

In production, you should monitor at least:

Metric	Meaning
Consumer lag	How far behind consumers are
Throughput	Messages per second
Error rate	How many messages fail
Retry count	How often retry logic is used
DLT count	How many messages are dead-lettered
Consumer rebalances	How often partitions are reassigned
Broker disk usage	Kafka storage pressure
Request latency	How fast Kafka responds
Under-replicated partitions	Replication health problem

A weak alert:

```
consumer_lag > 1000
```

A better alert:

```
consumer lag > 1000 for 10 minutes
```

An even better alert considers business impact:

```
payment-events consumer lag is growing for 15 minutes during business hours
```

Schema Evolution

Kafka messages usually have structure.

Version 1:

```
{
  "orderId": 123,
  "amount": 49.99
}
```

Version 2:

```
{
  "orderId": 123,
  "amount": 49.99,
  "currency": "USD"
}
```

That seems simple, but production systems may have old consumers still expecting the old format.

Schema evolution means changing event structure without breaking services.

Safe change:

```
{
  "orderId": 123,
  "amount": 49.99,
  "currency": "USD"
}
```

Adding an optional field is usually safe.

Risky change:

```
{
  "id": 123,
  "total": 49.99
}
```

Renaming fields can break consumers.

Very risky change:

```
{
  "orderId": "123"
}
```

Changing a field type from number to string can break deserialization.

Production guideline:

```
Events are contracts. Treat them like public APIs.
```

Backward Compatibility

Backward compatibility means a newer message can still be read by older consumers.

Old consumer expects:

```
{
  "orderId": 123,
  "amount": 49.99
}
```

New producer sends:

```
{
  "orderId": 123,
  "amount": 49.99,
  "currency": "USD"
}
```

The old consumer can ignore `currency` , so this is backward-compatible.

This is not backward-compatible:

```
{
  "orderId": 123,
  "totalAmount": 49.99
}
```

The old consumer expects `amount` , but it is gone.

Practical Java and Spring Boot Example

A basic Kafka listener might look like this:

```
@KafkaListener(topics = "orders", groupId = "order-service")
public void consume(OrderCreatedEvent event) {
    orderService.process(event);
}
```


That is fine for learning, but production code needs more thought:

```
@KafkaListener(topics = "orders", groupId = "order-service")
public void consume(OrderCreatedEvent event) {
    try {
        orderService.process(event);
    } catch (InvalidOrderException ex) {
        deadLetterPublisher.publish(event, ex);
    } catch (Exception ex) {
        throw ex;
    }
}
```

In real Spring Kafka systems, retry and dead-letter behavior is often configured using framework-level error handlers, not only manual `try/catch`.

Conceptually, the flow is:

```
Valid message -> process -> commit
Temporary failure -> retry
Permanent failure -> DLT -> commit
```

Production Checklist

Before calling a Kafka consumer production-ready, ask:

- Can it safely process duplicate messages?
- Does it retry temporary failures?
- Does it stop retrying permanent failures?
- Does it publish failed records to a dead-letter topic?
- Are failed messages easy to inspect?
- Is consumer lag monitored?
- Are alerts based on sustained problems, not tiny spikes?
- Are schemas versioned carefully?
- Can old consumers survive new message versions?
- Do dashboards show throughput, lag, errors, and retries?

Practice Exercises

Exercise 1: Basic Consumer Failure Handling

Create a Kafka consumer that reads `OrderCreated` events.

Practice behavior:

- If the event is valid, process it.
- If `orderId` is missing, treat it as invalid.
- If `amount <= 0`, reject it.
- Log the failure clearly.

Example event:

```
{
  "orderId": 101,
  "customerId": 55,
  "amount": 49.99
}
```

Goal: understand the difference between a valid event and a bad event.

Exercise 2: Retry Temporary Failures

Simulate a temporary database failure.

Example:

```
if (Math.random() < 0.3) {
  throw new RuntimeException("Database temporarily unavailable");
}
```

Practice:

- Retry failed processing.
- Add a maximum retry count.
- Add delay or backoff between retries.
- Log each retry attempt.

Goal: learn when retrying helps.

Exercise 3: Dead-Letter Topic

Create a dead-letter topic:

```
orders.DLT
```

Practice:

- Send invalid messages to the dead-letter topic.
- Include the original payload.
- Include the error reason.
- Include timestamp, topic, partition, and offset if available.

Goal: prevent one bad message from blocking the whole consumer.

Exercise 4: Poison Message Demo

Create one message that always fails.

Example:

```
{
  "orderId": null,
  "customerId": 55,
}
```

```
"amount": 49.99  
}
```

Practice:

- Observe what happens without DLT handling.
- Then add DLT handling.
- Confirm the consumer continues processing later messages.

Goal: understand why poison messages are dangerous in production.

Exercise 5: Consumer Lag Simulation

Create a producer that sends messages faster than the consumer can process them.

Example:

```
Producer: 100 messages/second  
Consumer: sleeps 200ms per message
```

Practice:

- Watch consumer lag increase.
- Speed up the consumer and watch lag decrease.
- Add another consumer instance if partitions allow it.

Goal: understand what consumer lag means.

Exercise 6: Partition and Consumer Group Experiment

Create a topic with 3 partitions.

Then test:

```
1 consumer  
2 consumers  
3 consumers  
5 consumers
```

Observe:

- With 1 consumer, it handles all partitions.
- With 3 consumers, each can handle one partition.
- With 5 consumers, 2 consumers sit idle.

Goal: understand Kafka parallelism.

Exercise 7: Throughput Dashboard

Track simple metrics in your app:

- Messages consumed
- Messages failed
- Messages retried
- Messages sent to DLT

- Average processing time

You can print them every 10 seconds first:

```
Consumed: 1200
Failed: 18
Retried: 12
DLT: 3
Average processing time: 45ms
```

Goal: build observability thinking before using full monitoring tools.

Exercise 8: Consumer Lag Alert Rule

Write alert rules in plain English or pseudo-config.

Examples:

```
Alert if consumer lag > 1000 for 10 minutes.
Alert if DLT messages > 10 in 5 minutes.
Alert if retry rate > 20% for 15 minutes.
```

Goal: learn what production alerts should actually detect.

Exercise 9: Schema Evolution Practice

Start with this event:

```
{
  "orderId": 101,
  "amount": 49.99
}
```

Then make safe and unsafe changes.

Safe:

```
{
  "orderId": 101,
  "amount": 49.99,
  "currency": "USD"
}
```

Unsafe:

```
{
  "id": 101,
  "total": 49.99
}
```

Practice:

- Add an optional field.
- Rename a field.
- Remove a field.
- Change a field type.

Goal: learn how schema changes can break consumers.

Exercise 10: Idempotent Consumer

Create a consumer that writes processed order IDs into a database or in-memory set.

If the same order appears twice, skip duplicate processing.

Example:

```
Order 101 processed.  
Order 101 already processed. Skipping duplicate.
```

Goal: handle Kafka's at-least-once delivery behavior safely.

Recommended Practice Sequence

1. Basic consumer validation
2. Retry temporary failures
3. Dead-letter topic
4. Poison message handling
5. Consumer lag simulation
6. Partition and consumer group scaling
7. Metrics and alerts
8. Schema evolution
9. Idempotent consumer

Lab: Kafka Resilience and Observability

Build a small **Order Event Processing** system with Kafka, Java, and Spring Boot.

Lab Goal

Create a Kafka consumer that can:

- Read order events
- Retry temporary failures
- Send bad messages to a dead-letter topic
- Track simple processing metrics
- Demonstrate consumer lag
- Handle duplicate messages safely

Topics

```
orders  
orders.DLT
```

Event Examples

Valid message:

```
{
  "orderId": 101,
  "customerId": 55,
  "amount": 49.99,
  "currency": "USD"
}
```

Invalid message:

```
{
  "orderId": null,
  "customerId": 55,
  "amount": -10
}
```

Part 1: Create Producer

Create a producer that sends order events to:

```
orders
```

Send at least:

- 5 valid orders
- 2 invalid orders
- 1 duplicate order
- 1 message that simulates temporary failure

Part 2: Create Consumer

Create a consumer that reads from `orders` .

Validation rules:

```
orderId must not be null
customerId must not be null
amount must be greater than 0
currency must not be blank
```

If valid:

```
Process order
Mark orderId as processed
Commit successfully
```

If duplicate:

```
Skip processing  
Log duplicate detected
```

Part 3: Add Retry Logic

Simulate temporary failure for one order.

Example:

```
if (event.getOrderId().equals(500L)) {  
    throw new RuntimeException("Temporary payment service failure");  
}
```

Expected behavior:

```
Attempt 1 failed  
Attempt 2 failed  
Attempt 3 failed  
Send to DLT
```

Part 4: Dead-Letter Topic

Invalid messages and exhausted retry messages should be sent to:

```
orders.DLT
```

DLT payload should include:

```
{  
  "originalMessage": {},  
  "errorReason": "amount must be greater than 0",  
  "failedAt": "timestamp"  
}
```

Part 5: Observe Consumer Lag

Make the consumer slow:

```
Thread.sleep(2000);
```

Then send many messages quickly.

Observe:

```
Consumer falls behind  
Lag increases  
Consumer catches up after producer stops
```

Part 6: Add Basic Metrics

Print metrics every few messages:

```
Processed: 10
Failed: 2
Retried: 3
Sent to DLT: 2
Duplicates skipped: 1
```

Expected Final Output

Your console should show something like:

```
Received order 101
Order 101 processed successfully

Received order 102
Order 102 processed successfully

Received order null
Validation failed: orderId is required
Sent message to orders.DLT

Received order 500
Processing failed. Retry attempt 1
Processing failed. Retry attempt 2
Processing failed. Retry attempt 3
Sent message to orders.DLT

Received order 101
Duplicate order detected. Skipping.
```

Completion Checklist

You are done when you can show:

- Valid messages are processed
- Invalid messages go to DLT
- Temporary failures are retried
- Failed retries eventually go to DLT
- Duplicate order IDs are skipped
- Consumer lag can be created and explained
- Metrics are printed or exposed

Challenge Extension

Add one more topic:

```
orders.retry
```


Flow:

```
orders -> retry -> orders.DLT
```

This makes the lab closer to a real production Kafka design.

Quick Review

Kafka production readiness is about resilience and visibility.

Resilience means:

```
The system keeps working even when some messages or dependencies fail.
```

Observability means:

```
The team can see what is happening before users complain.
```

The core ideas from this module are:

- Use retries for temporary failures.
- Use dead-letter topics for messages that cannot be processed.
- Watch consumer lag closely.
- Monitor throughput and error rates.
- Treat message schemas as contracts.
- Design consumers to tolerate duplicates.